

핸즈온 랩: Spark와 Microsoft Fabric Notebook을 사용하여 데이터 수집하기

이 랩에서는 Microsoft Fabric Notebook을 생성하고 PySpark를 사용하여 Azure Blob Storage 경로에 연결한 다음, 쓰기 최적화를 사용하여 데이터를 Lakehouse로 로드합니다.

이 실습을 완료하는 데 약 **30분**이 소요됩니다.

이 경험을 위해, 여러 Notebook 코드 셀에 걸쳐 코드를 작성하게 되는데, 이는 실제 환경에서 작업하는 방식과는 다를 수 있지만 디버깅에 유용할 수 있습니다.

또한 샘플 데이터 세트로 작업하기 때문에, 최적화가 대규모 프로덕션 환경에서 볼 수 있는 것과 같지는 않지만, 여전히 개선 효과를 볼 수 있으며 매 밀리초가 중요할 때 최적화는 핵심입니다.

참고: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

Workspace 만들기

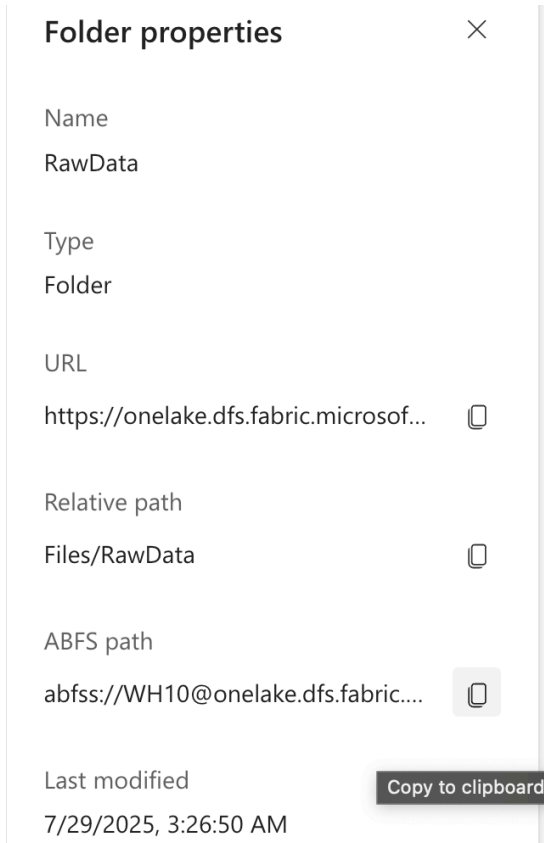
Fabric에서 데이터 작업을 시작하기 전에 Fabric 평가판이 활성화된 Workspace를 만들어야 합니다.

- 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) `https://app.fabric.microsoft.com/home?experience=fabric` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
- 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
- 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial*, *Premium*, 또는 *Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
- 새 Workspace가 열리면 처음에는 비어 있어야 합니다.

Workspace 및 Lakehouse 목적지 만들기

먼저 새 Lakehouse를 만들고, Lakehouse에 목적지 폴더를 만듭니다.

- Workspace에서 **+ New item > Lakehouse**를 선택하고, 이름을 지정한 후 **Create**를 선택합니다.
- Files**에서 **[...]**를 선택하여 `RawData` 라는 이름의 **New subfolder**를 만듭니다.
- Lakehouse 내의 Lakehouse Explorer에서 **RawData > ... > Properties**를 선택합니다.



4. **RawData** 폴더의 **ABFS path**를 나중에 사용할 수 있도록 빈 메모장에 복사합니다. 경로는 다음과 유사한 형태여야 합니다.

```
abfss://{workspace_name}@onelake.dfs.fabric.microsoft.com/{lakehouse_name}.Lakehouse/Files/{folder_name}
```

이제 Lakehouse와 RawData 목적지 폴더가 있는 Workspace가 준비되었습니다.

Fabric Notebook 생성 및 외부 데이터 로드하기

핸즈온의 의미: 이 단계는 Fabric Notebook의 핵심 기능 중 하나를 보여줍니다. 즉, 외부 클라우드 스토리지(여기서는 공개 Azure Blob Storage)에 저장된 대용량 데이터(NYC 옐로우 택시 데이터)에 직접 연결하여 데이터를 Spark DataFrame으로 읽어오는 과정을 경험합니다. 이를 통해 데이터를 먼저 Fabric으로 이동시키지 않고도 원격 데이터를 직접 분석하고 처리할 수 있는 강력함을 체험할 수 있습니다.

1. Lakehouse의 상단 메뉴에서 **Open notebook > New notebook**을 선택하여 새 Notebook을 엽니다.
2. 기본 셀에서 코드가 **PySpark (Python)**으로 설정되어 있는지 확인합니다.
3. 코드 셀에 다음 코드를 삽입합니다. 이 코드는 다음 작업을 수행합니다.
 - 연결 문자열을 위한 매개변수 선언
 - 연결 문자열 구축
 - 데이터를 DataFrame으로 읽기

```
# Azure Blob Storage access info
blob_account_name = "azureopendatastorage"
blob_container_name = "nyctlc"
blob_relative_path = "yellow"

# Construct connection path
wasbs_path =
f'wasbs://{blob_container_name}@{blob_account_name}.blob.core.windows.net/{blob_relative_path}'
print(wasbs_path)

# Read parquet data from Azure Blob Storage path
blob_df = spark.read.parquet(wasbs_path)
```

코드 설명:

- `wasbs://...`: Azure Blob Storage에 연결하기 위한 프로토콜 및 경로 형식입니다. `wasbs` 는 보안 연결을 의미합니다.
- `spark.read.parquet(wasbs_path)`: Spark 세션을 사용하여 지정된 `wasbs_path` 의 모든 Parquet 파일을 읽어 하나의 Spark DataFrame(`blob_df`)으로 로드합니다.

4. 코드 셀 옆의 ▶ **Run Cell**을 선택하여 DataFrame에 데이터를 연결하고 읽습니다.

예상 결과: 명령이 성공적으로 실행되고

`wasbs://nyctlc@azureopendatastorage.blob.core.windows.net/yellow`가 출력됩니다.

5. 데이터를 파일로 쓰려면 이제 **RawData** 폴더의 **ABFS** 경로가 필요합니다.

6. 새 코드 셀에 다음 코드를 삽입합니다.

```
# Declare file name
file_name = "yellow_taxi"

# Construct destination path
output_parquet_path = f"***InsertABFSPathHere**/{file_name}"
print(output_parquet_path)

# Load the first 1000 rows as a Parquet file
blob_df.limit(1000).write.mode("overwrite").parquet(output_parquet_path)
```

코드 설명:

- `f"***InsertABFSPathHere**/{file_name}"`: 이전에 복사해 둔 `RawData` 폴더의 ABFS 경로를 `***InsertABFSPathHere **` 부분에 붙여넣습니다.
- `blob_df.limit(1000)`: 전체 데이터가 매우 크므로, 테스트를 위해 처음 1000개의 행만 선택합니다.
- `.write.mode("overwrite").parquet(output_parquet_path)`: 선택된 데이터를 Parquet 형식으로 지정된 `output_parquet_path`에 저장합니다. `mode("overwrite")`는 동일한 이름의 파일이 이미 존재할 경우 덮어쓰도록 합니다.

7. **RawData**의 ABFS 경로를 추가하고 ▶ **Run Cell**을 선택하여 1000개의 행을 `yellow_taxi.parquet` 파일로 씁니다.

8. Lakehouse Explorer에서 데이터 로드를 확인하려면 **Files > ... > Refresh**를 선택합니다.

이제 **RawData**라는 새 폴더와 그 안에 `yellow_taxi.parquet` "파일"이 보일 것입니다. (실제로는 파티션 파일들이 들어있는 폴더로 표시됩니다.)

데이터 변환 및 Delta 테이블로 로드하기

데이터 수집 작업은 단순히 파일을 로드하는 것만으로 끝나지 않을 가능성이 높습니다. Lakehouse의 Delta 테이블은 확장 가능하고 유연한 쿼리 및 저장을 허용하므로, 이 테이블도 만들어 보겠습니다.

핸즈온의 의미: 이 단계는 일반적인 **ETL(Extract-Transform-Load)** 프로세스 중 **T(변환)**와 **L(로드)** 단계를 수행합니다. 원시 파일(Parquet)을 읽어와서, 비즈니스 요구사항에 따라 데이터를 변환(새 열 추가, 불필요한 데이터 필터링)한 다음, 최종적으로 분석에 최적화된 구조화된 **Delta 테이블**로 저장하는 과정을 경험합니다.

1. 새 코드 셀을 만들고 다음 코드를 삽입합니다.

```
from pyspark.sql.functions import col, to_timestamp, current_timestamp, year, month

# Read the parquet data from the specified path
raw_df = spark.read.parquet(output_parquet_path)

# Add dataload_datetime column with current timestamp
filtered_df = raw_df.withColumn("dataload_datetime", current_timestamp())

# Filter columns to exclude any NULL values in storeAndFwdFlag
filtered_df = filtered_df.filter(col("storeAndFwdFlag").isNotNull())

# Load the filtered data into a Delta table
table_name = "yellow_taxi"
filtered_df.write.format("delta").mode("append").saveAsTable(table_name)

# Display results
display(filtered_df.limit(1))
```

코드 설명:

- `raw_df.withColumn("dataload_datetime", current_timestamp())`: 데이터가 로드된 시점을 기록하기 위해 `dataload_datetime`이라는 새 열을 현재 타임스탬프 값으로 추가합니다.
- `filtered_df.filter(col("storeAndFwdFlag").isNotNull())`: `storeAndFwdFlag` 열에 Null 값이 있는 행을 제거하여 데이터를 정제합니다.
- `.write.format("delta").mode("append").saveAsTable(table_name)`: 필터링된 데이터를 `yellow_taxi`라는 이름의 Delta 테이블로 저장합니다. `mode("append")`는 테이블이 이미 존재할 경우 데이터를 추가하도록 합니다.

2. 코드 셀 옆의 ▶ **Run Cell**을 선택합니다.
3. 표시된 결과를 검토하고 확인합니다.

```
16 # Display results
17 display(filtered_df.limit(1))
```

✓ 16 sec - Command executed in 16 sec 689 ms by 이정환 on 3:41:30 AM, 7/29/25

> Spark jobs (6 of 6 succeeded)  Resources  Log

 Table  New chart

Table view

	ABC vendorID	 tpepPickupDateTime	 tpepDropoffDateTime	123 passengerCount	12 tripDistance	ABC puLocationId
1	CMT	2011-04-04 10:30:39	2011-04-04 10:37:22	1	1.1	NULL

이제 외부 데이터에 성공적으로 연결하고, Parquet 파일로 쓰고, 데이터를 DataFrame으로 로드하고, 데이터를 변환하고, Delta 테이블로 로드했습니다.

SQL 쿼리로 Delta 테이블 데이터 분석하기

이 랩은 데이터 수집에 중점을 두고 *추출, 변환, 로드* 프로세스를 설명하지만, 데이터를 미리 보는 것도 중요합니다.

개념 설명: Spark SQL

Spark SQL은 Spark의 SQL 언어 API로, SQL 문을 실행하거나 데이터를 관계형 테이블에 유지하는 데 사용할 수 있습니다. 많은 데이터 분석가들이 SQL 구문에 익숙하기 때문에, PySpark 코드 내에서 직접 SQL을 사용하여 데이터를 쿼리하고 조작할 수 있는 강력하고 유연한 방법을 제공합니다. `createOrReplaceTempView`는 DataFrame을 SQL로 쿼리할 수 있는 임시 뷰(View)로 등록하는 함수입니다.

1. 새 코드 셀을 만들고 아래 코드를 삽입합니다.

```
# Load table into df
delta_table_name = "yellow_taxi"
table_df = spark.read.format("delta").table(delta_table_name)

# Create temp SQL table
table_df.createOrReplaceTempView("yellow_taxi_temp")

# SQL Query
table_df = spark.sql('SELECT * FROM yellow_taxi_temp')

# Display 10 results
display(table_df.limit(10))
```

코드 설명:

- `spark.read.format("delta").table(delta_table_name)`: `yellow_taxi` Delta 테이블을 읽어 DataFrame으로 로드합니다.
- `table_df.createOrReplaceTempView("yellow_taxi_temp")`: 이 DataFrame을 `yellow_taxi_temp` 라는 이름의 임시 뷰로 등록합니다. 이 뷰는 현재 Spark 세션에서만 유효합니다.
- `spark.sql('SELECT * FROM yellow_taxi_temp')`: PySpark 코드 내에서 SQL 쿼리를 직접 실행하여 `yellow_taxi_temp` 뷰에서 모든 데이터를 선택합니다.

2. 코드 셀 옆의 > **Run Cell**을 선택합니다.

추가 Hands-on

데이터 분석 - 자율 핸즈온

Microsoft Fabric 또는 Databricks 기반의 Spark 환경에서 NYC Yellow Taxi 데이터를 분석

예시: Microsoft Fabric 또는 Databricks 기반의 Spark 환경에서 NYC Yellow Taxi 데이터를 분석하여 "위험한(사기 가능성이 높은)" 운행 기록을 식별

- Delta 테이블에서 NYC Taxi 데이터를 로드
- 의심스러운 패턴을 분석 (예: 너무 짧은 거리인데 요금이 너무 높거나, 승차·하차 시간이 비정상인 경우 등)
- 위험도 기준을 정하고, 의심스러운 운행 기록 필터링
- 결과를 확인하고 Parquet 파일이나 테이블로 저장
-

조건	설명
너무 짧은 거리인데 요금이 비정상적으로 높은 경우	거리 < 0.5 마일 & 총 요금 > \$50
운행 시간이 0이거나 너무 긴 경우	dropoff - pickup 시간이 0분 또는 > 3시간
요금이 음수	<code>fare_amount < 0</code>
승차/하차 좌표가 동일하거나 이상함	동일한 lat/lon 값 등

```
1  %%sql
2
3  SELECT *
4  FROM yellow_taxi_temp
5  WHERE
6    -- (1) 너무 짧은 거리인데 요금이 매우 높은 경우
7    (tripDistance < 0.5 AND totalAmount > 50)
8
9    OR
10
11    -- (2) 운행 시간이 0분이거나 3시간 초과일 경우
12    (tpepPickupDateTime = tpepDropoffDateTime) OR
13    (unix_timestamp(tpepDropoffDateTime) - unix_timestamp(tpepPickupDateTime)) > 3 * 60 * 60
14
15    OR
16
17    -- (3) 요금이 음수인 경우
18    (fareAmount < 0)
19
20    OR
21
22    -- (4) 출발/도착 좌표가 동일한 경우
23    (startLon = endLon AND startLat = endLat)
```

2 sec - Command executed in 2 sec 472 ms by 이정환 on 4:04:17 AM, 7/29/25

Spark jobs (2 of 2 succeeded) Resources Log

Table + New chart 24 columns, 17

Table view Download Filter by keyword

	ABC vendorID	tpepPickupDateTime	tpepDropoffDateTime	123 passengerCount	12 tripDistance	ABC puLocationId	ABC doLocationId	12 startLon	12 startLat	12 endLon
1	CMT	2011-04-08T12:04:07Z	2011-04-08T12:04:07Z	1	7.6	NULL	NULL	-73.98304	40.750684	-73.98304
2	CMT	2011-04-10T15:15:21Z	2011-04-10T15:57:46Z	1	11.7	NULL	NULL	-73.872384	40.774183	-73.872384
3	CMT	2011-03-31T23:56:34Z	2011-04-01T00:00:13Z	1	0.8	NULL	NULL	0	0	0
4	CMT	2011-04-02T11:34:09Z	2011-04-02T11:40:21Z	1	1.5	NULL	NULL	0	0	0
5	CMT	2011-04-08T11:54:45Z	2011-04-08T12:28:38Z	1	0	NULL	NULL	-75.3359633...	40.64866892...	-73.9826998...

머신러닝 기반 이상치 탐지 헨즈온 (IsolationForest)

실습 목표

- Spark DataFrame → Pandas DataFrame (샘플)
- Scikit-learn 기반 IsolationForest 이상치 탐지 모델 적용
- 이상치 결과를 다시 Spark로 변환하여 시각화 또는 저장

분석 대상 컬럼 전처리

```
# 사용할 컬럼 선택 (수치형 위주)

selected_columns = [
    'tripDistance', 'fareAmount', 'tipAmount', 'tollsAmount', 'totalAmount',
    'startLon', 'startLat', 'endLon', 'endLat'
]

# Null 제거 및 샘플링 (예: 10,000건)
sample_df = table_df.select(selected_columns).dropna().limit(10000)

# Pandas로 변환 (학습용)
pandas_df = sample_df.toPandas()
```

IsolationForest 이상치 탐지 모델 적용

```
from sklearn.ensemble import IsolationForest

# Isolation Forest 모델 정의
model = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)

# 모델 학습
model.fit(pandas_df)

# 이상치 예측 (결과: 1은 정상, -1은 이상치)
pandas_df['anomaly'] = model.predict(pandas_df)

# 이상치만 필터링
anomalies_df = pandas_df[pandas_df['anomaly'] == -1]
```

다시 Spark로 변환하여 시각화 또는 저장

```
# Pandas → Spark DataFrame

anomaly_spark_df = spark.createDataFrame(anomalies_df)

# 결과 확인
display(anomaly_spark_df.limit(10))
```

결과 예시:

```
1 # Pandas → Spark DataFrame
2 anomaly_spark_df = spark.createDataFrame(anomalies_df)
3
4 # 결과 확인
5 display(anomaly_spark_df.limit(10))
```

✓ <1 sec - Command executed in 946 ms by 이정환 on 5:22:24 AM, 7/29/25

	12 tripDistance	12 fareAmount	12 tipAmount	12 tollsAmount	12 totalAmount	12 startLon	12 startLat	12 endLon	12 endLat	12L anomaly
1	17.3	45.0	12.57	4.8	62.87	-73.783524	40.648711	-73.966273	40.753263	-1
2	17.3	45.0	7.5	4.8	57.8	-73.7832	40.648623	-73.982695	40.745242	-1
3	0.8	4.5	0.0	0.0	5.5	0.0	0.0	0.0	0.0	-1
4	28.9	100.0	10.08	6.0	116.08	-73.975602	40.760748	-74.076672	41.042262	-1
5	1.5	6.1	0.0	0.0	6.6	0.0	0.0	0.0	0.0	-1
6	17.1	45.0	5.0	4.8	55.3	-73.785948	40.644185	-73.972206	40.75652	-1
7	0.0	45.0	0.0	4.8	50.3	-75.3359633...	40.64866892...	-73.9826998...	40.7553473...	-1
8	1.8	7.7	1.74	0.0	10.44	0.0	0.0	0.0	0.0	-1
9	1.1	5.3	0.0	0.0	5.8	0.0	0.0	0.0	0.0	-1
10	0.0	12.5	1.0	0.0	14.5	-74.6898307...	40.19056355...	-74.6898307...	40.1905635...	-1

추가 목표: 이상치 레코드에 대해 전체 컬럼 조회 + 테이블로 저장

1 이상치 판별에 사용한 원본 Row ID 확보

Isolation Forest를 학습한 `pandas_df`에는 원래의 Row ID가 없기 때문에, 식별을 위해 샘플링 단계에서 Row 번호를 부여해 둡니다:

```
# Pandas 전환 전, 인덱스 부여
sample_df = (
    table_df.selectExpr("monotonically_increasing_id() as row_id", *selected_columns)
    .dropna()
    .limit(10000)
)

# Pandas로 변환
pandas_df = sample_df.toPandas()
```

2 이상치 예측 후 Row ID 기준으로 전체 데이터 조인

```
# 이상치 필터링
anomalies_df = pandas_df[model.predict(pandas_df[selected_columns]) == -1]

# row_id만 추출
anomaly_ids = anomalies_df['row_id'].tolist()
```



```
import pyspark.sql.functions as F

# Spark에서 원본 테이블과 row_id 기준으로 필터링
sample_full_df = (
    table_df.withColumn("row_id", F.monotonically_increasing_id())
)

from pyspark.sql.functions import col

# 이상치만 필터링
suspicious_full_df = sample_full_df.filter(col("row_id").isin(anomaly_ids))
```

`F` 는 일반적으로 PySpark 함수들을 불러올 때 사용하는 축약형 이름입니다.

다음과 같이 먼저 `pyspark.sql.functions` 를 `F` 라는 이름으로 `import`

리소스 정리

이 실습에서는 Fabric에서 PySpark와 함께 Notebook을 사용하여 데이터를 로드하고 Parquet으로 저장했습니다. 그런 다음 해당 Parquet 파일을 사용하여 데이터를 추가로 변환했습니다. 마지막으로 SQL을 사용하여 Delta 테이블을 쿼리했습니다.

탐색을 마쳤으면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

1. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
2. **Workspace settings**를 선택하고 **General** 섹션에서 아래로 스크롤하여 **Remove this workspace**를 선택합니다.
3. **Delete**를 선택하여 Workspace를 삭제합니다.